Entropy and Cross-Entropy at Inference Time

Setting

A model with parameters θ takes an input x and gives a probability distribution $q_\theta(k \mid x)$ over the classes $k = 1, \dots, K$. In this note, all logarithms are natural logarithms, so all quantities are measured in nats. If we use $\log_2$ instead, we get bits. For example, a fair coin has entropy $\ln 2 \approx 0.693$ nats, which is exactly 1 bit.

In the cross-entropy formula, p is the *target* distribution and q is the model's *predicted* distribution. The target is what we compare against; the prediction is what the model gives us. We will see below what the target looks like for a coin and for labelled data.

What does entropy measure?

In this section we use $\log_2$, so entropy is measured in bits. At the end of the section we explain how bits and nats are related.

Let us start with a fair coin, which lands Heads with probability 0.5 and Tails with probability 0.5. Its entropy is

$$H = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 0.5 + 0.5 = 1 \text{ bit per toss.} \tag{1}$$

This matches our common sense. Before a fair toss, both outcomes are equally likely, so we have no way to guess better than chance. Writing Heads as 1 and Tails as 0, we need exactly one bit per toss, and no lossless method can do better on average. For a coin, 1 bit is the largest possible entropy: a fair coin is as unpredictable as a coin can be.

Now take a coin that lands Heads with probability 0.6 and Tails with probability 0.4. Its entropy is

$$H = -0.6 \log_2 0.6 - 0.4 \log_2 0.4 \approx 0.971 \text{ bits per toss.} \tag{2}$$

This does not mean that one toss can be recorded using a fraction of a bit: recording a single Heads or Tails result requires one bit. Entropy is an average over many tosses. If we encode a long sequence of independent tosses using an efficient lossless method, the average length can approach 0.971 bits per toss, but it cannot go below that. The coin is slightly predictable (Heads comes up more often), and a good encoding takes advantage of this.

Bits and nats measure the same thing

Bits and nats are two units for the same quantity, just like kilometres and miles are two units for the same distance. Changing the base of the logarithm only multiplies every logarithm by the same constant:

$$\log_2 a = \frac{\ln a}{\ln 2} \qquad \text{for every } a > 0. \tag{3}$$

Since entropy is a sum of such logarithms, the whole entropy gets multiplied by the same constant. So

$$H \text{ in nats} = (H \text{ in bits}) \times \ln 2, \qquad H \text{ in bits} = \frac{H \text{ in nats}}{\ln 2}, \tag{4}$$

where $\ln 2 \approx 0.693$. Our two coins in both units:

Coin	Entropy in bits	Entropy in nats
Fair, $(0.5,\ 0.5)$	1	$\ln 2 \approx 0.693$
Biased, $(0.6,\ 0.4)$	≈ 0.971	≈ 0.673

The ordering never changes: whichever unit we use, the biased coin has less entropy than the fair coin. Only the numbers change.

Which unit should we use? Bits are easier to picture when we talk about encoding, because a bit is one 0 or 1. That is why we used $\log_2$ in this section. But in deep learning we use ln: libraries such as PyTorch compute cross-entropy with ln, so the loss values you see during training are in nats, and the derivative of $\ln z$ is simply $1/z$, which keeps the gradients simple. So from the next section onwards, we go back to ln and nats.

Inference

In production, θ is fixed. For every input x, the model gives $q_\theta(\cdot \mid x)$, and the system uses it either to make a prediction (for example, pick the class with the highest probability) or to sample an outcome. Ordinary prediction does not need any loss to be computed, and θ is not updated.

Entropy of a prediction

The output distribution has an entropy:

$$H(q_\theta(\cdot \mid x)) = -\sum_{k=1}^{K} q_\theta(k \mid x)\ \ln q_\theta(k \mid x). \tag{5}$$

To compute this, we need only the model's output. We do not need any label. It tells us how spread out the prediction is for this particular input. If all the probability sits on

one class, the entropy is zero. If the prediction is uniform over all K classes, the entropy takes its largest value, $\ln K$.

So an application can compute (5) for each input and use it as a confidence signal. For example, predictions with high entropy can be sent to a human for review. But please note: low entropy only means the model is confident. It does not mean the model is correct.

Cross-entropy

Cross-entropy compares the model's distribution q with a target distribution p:

$$H(p, q) = -\sum_{k} p(k)\ \ln q(k). \tag{6}$$

First, the coin

For a coin, we can write down p fully. Suppose we toss a fair coin before a cricket match, so $p = (0.5,\ 0.5)$ for (heads, tails). Suppose our model wrongly says the coin is biased, $q = (0.98,\ 0.02)$. Then

$$H(p, q) = -0.5\ \ln 0.98\ -\ 0.5\ \ln 0.02 \approx 0.010 + 1.956 = 1.966 \text{ nats}. \tag{7}$$

If the model were correct, that is $q = p$, we would get only $\ln 2 \approx 0.693$ nats. The extra $1.966 - 0.693 = 1.273$ nats is the price we pay for using the wrong model. This extra amount is the KL divergence, $\mathrm{KL}(p \,\|\, q)$.

We can get the same number 1.966 from data, without writing p down at all. Toss the coin many times. For each toss, look at the face that actually came up, take $-\ln$ of the probability the model gave to that face, and then take the average over all tosses. As the number of tosses grows, this average comes closer and closer to $H(p, q)$. So p enters only through what we actually observed.

Now, labelled data

With labelled data, the situation is a little different. For each input x we see only one outcome, namely its label y. We do not know the full distribution of labels for that x. So, as the target, we use the *one-hot* distribution on y: probability 1 on class y and 0 on every other class. With this target, (6) becomes

$$\ell(x, y; \theta) = -\ln q_\theta(y \mid x). \tag{8}$$

In simple words: the loss for one example is $-\ln$ of the probability the model gave to the correct label.

Over a labelled set $\{(x_i, y_i)\}_{i=1}^{N}$, the reported loss is the average

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} -\ln q_\theta(y_i \mid x_i), \tag{9}$$

which is the average cross-entropy between each one-hot target and the model's prediction for that input.

Notice that (9) is exactly the same recipe as the coin: look at what actually happened, take $-\ln$ of the probability the model gave to it, and average. Averaging over many inputs here plays the same role that averaging over many tosses played for the coin.

Why p and q cannot be swapped

The order of p and q in (6) matters. Suppose p is one-hot on y and we swap the roles, giving the reversed expression

$$-\sum_k q(k)\ \ln p(k). \tag{10}$$

This is infinite whenever q gives positive probability to any class other than y, because $p(k) = 0$ for those classes and $\ln 0 = -\infty$.[1]

Training, evaluation, and inference

- **Training** minimises $\mathcal{L}(\theta)$ in (9) with respect to θ. This changes q_θ.
- **Evaluation** computes the same $\mathcal{L}(\theta)$ on held-out labelled data, keeping θ fixed. Computing a loss does not by itself mean we are training.
- **Inference** uses $q_\theta(\cdot \mid x)$ to predict or to sample. At that moment the true labels are usually not available, so label-based cross-entropy cannot be computed then. But the entropy (5) of each prediction can always be computed. If labels come in later, computing $\mathcal{L}(\theta)$ on them tells us whether the fixed model still fits the new data.

[1] Here we use the usual convention $0 \cdot \ln 0 = 0$. A model that ends with a softmax gives $q(k) > 0$ for every class, so in practice the reversed expression is always infinite.